BTEUP EXAM NOTES | ENGLISH EDITION

Mobile App Development

Unit I: Introduction to Android: Platform, SDK, First App, Manifest

About these notes

Part A explains every topic in very simple English with real-life examples, so a beginner can understand it from zero.
Part B gives important exam questions with full 5-mark model answers, ready to write in the exam.
At the end: quick revision sheet, common mistakes, and a self-test.

Syllabus covered (Unit I)

Introduction to Android: The Android Platform, Android SDK, Eclipse Installation, Android Installation, Building your First Android application, Understanding Anatomy of Android Application, Android Manifest file.

Unit Overview and Contents

Simple idea: Android is the operating system on most mobile phones. To build an app for it, you need to know (1) **how Android is built** (platform), (2) **the tools** (SDK and IDE), (3) **how to set up** your computer and phone, (4) **how to make a first app**, and (5) **what files an app contains**, especially the **AndroidManifest.xml**, which is the app's ID card.

Important note about Eclipse

Your syllabus says **Eclipse**, so the Eclipse steps are included. But Google **ended official support for Eclipse ADT (Android Developer Tools) in 2016**. **Android Studio** is now the official IDE. So these notes give **both**: Eclipse + ADT (for the exam, as per syllabus) and Android Studio (what you should use in practice).

Topic	Exam importance	Where in notes
Android platform and architecture	Very High	Part A: Topic 1 | Part B: Q1, Q2, Q12
Android SDK	Very High	Part A: Topic 2 | Part B: Q3
Eclipse and Android installation	High	Part A: Topic 3, 4 | Part B: Q4, Q5, Q6
First Android application	Very High	Part A: Topic 5 | Part B: Q7
Anatomy of Android application	Very High	Part A: Topic 6 | Part B: Q8, Q9, Q11
AndroidManifest.xml	Very High	Part A: Topic 7 | Part B: Q10

Contents

Part A: Easy Explanation	Part B: Important Questions with Answers
Topic 1: The Android Platform Topic 2: Android SDK Topic 3: Eclipse and Android Studio installation Topic 4: Android installation (emulator, phone) Topic 5: First Android application Topic 6: Anatomy of an Android application Topic 7: AndroidManifest.xml	Q1 Android: features and versions (Very Likely) Q2 Android architecture (Very Likely) Q3 Android SDK (Very Likely) Q4 Android Studio installation (Likely) Q5 Eclipse + ADT installation (Likely) Q6 AVD, emulator, run on phone (Likely) Q7 First Android application (Very Likely) Q8 Anatomy of Android application (Very Likely) Q9 Build process, R.java, resources (Likely) Q10 AndroidManifest.xml (Very Likely) Q11 Components of an app (Likely) Q12 Dalvik vs ART, APK vs AAB (Possible)

How to study this unit (suggested time: 1 day)

1) Read Part A once to understand. 2) Practice the architecture diagram, project structure and manifest until you can draw them without looking. 3) Revise using the Quick Revision Sheet. 4) Attempt the Self-Test without looking.

PART A: Easy Explanation (Simple English)

Topic 1: The Android Platform

Real-life example

A smartphone is like a restaurant. The **operating system (Android)** is the kitchen and manager that run everything. The **apps** are the dishes the customer orders. To cook new dishes (apps), you must know how the kitchen works.

Definition

Android is an open-source, Linux-based mobile operating system developed by **Google** and the **Open Handset Alliance (OHA)**, used on smartphones, tablets, smart TVs, watches and cars.

Quick history

- **2003:** Android Inc. was founded (Andy Rubin and others). **2005:** Google bought it.
- **2007:** Open Handset Alliance was formed. **2008:** first Android phone, **HTC Dream** (Android 1.0).
- Android versions were named after **desserts** (Cupcake, Donut ... Pie). From Android 10, only numbers are used.

Version	Name	API level
1.5	Cupcake	3
2.2	Froyo	8
2.3	Gingerbread	9-10
4.0	Ice Cream Sandwich	14-15
4.1 - 4.3	Jelly Bean	16-18
4.4	KitKat	19
5.0 - 5.1	Lollipop (ART becomes default)	21-22
6.0	Marshmallow (runtime permissions)	23
7.0 - 7.1	Nougat	24-25
8.0 - 8.1	Oreo	26-27
9	Pie	28
10 / 11 / 12	(no dessert name)	29 / 30 / 31
13 / 14 / 15 / 16	(no dessert name)	33 / 34 / 35 / 36

Remember

API level is the number that identifies the Android platform version for developers (for example Android 5.0 = API 21). Your app declares the lowest API level it supports (**minSdk**). Check developer.android.com for the newest release.

Features of Android

- **Open source and free:** manufacturers can use and customise it.
- **Rich UI and multitasking** with widgets, notifications, split screen.
- **Connectivity:** Wi-Fi, Bluetooth, NFC, GPS, 4G/5G.

- **Storage:** SQLite database, files, shared preferences.
- **Hardware access:** camera, sensors, microphone, fingerprint.
- **Apps in Java or Kotlin** (Kotlin is Google's preferred language), distributed via Google Play.
- **Security:** every app runs in its own sandbox and needs permissions.

Android Architecture (Platform stack)

Android is built in **layers**. Each layer uses the services of the layer below it. Think of a multi-storey building: the foundation is the Linux kernel and the top floor is the apps.

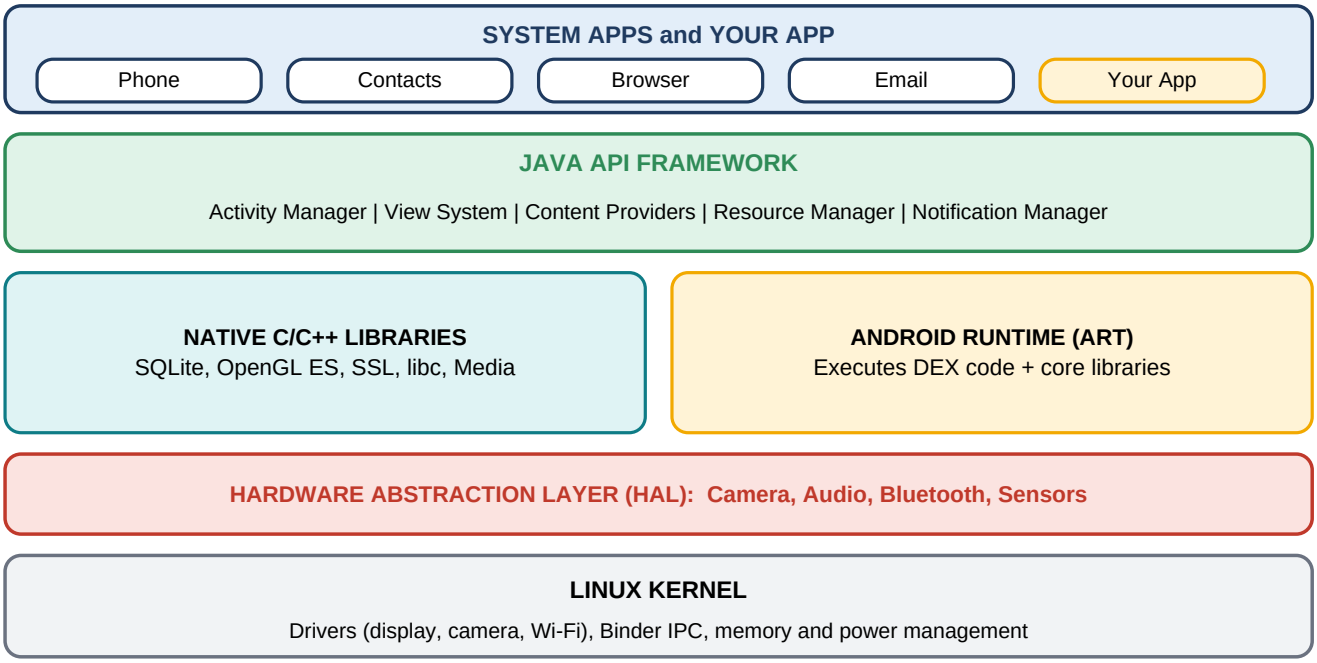


Figure 1: Android software stack (architecture)

Layer (top to bottom)	What it does
System apps	Built-in apps (phone, contacts, browser, email). Your app sits at this level too.
Java API framework	The APIs you use to build apps: Activity Manager (life cycle), View System (UI), Content Providers (share data), Resource Manager, Notification Manager.
Native C/C++ libraries	Core libraries written in C/C++ such as SQLite (database), OpenGL ES (graphics), SSL (security), libc, media libraries.
Android Runtime (ART)	Runs the app's DEX code; each app runs in its own process with its own ART instance. Includes core Java libraries. Before Android 5.0 the runtime was Dalvik.
Hardware Abstraction Layer (HAL)	Gives standard interfaces to hardware (camera, Bluetooth, audio, sensors) so higher layers need not know hardware details.
Linux kernel	Foundation: device drivers, memory, process and power management, security, networking.

Textbook note

Many textbooks show a 4-layer view: **Applications, Application Framework, Libraries + Android Runtime, Linux Kernel**. It is the same stack. Google's current documentation also shows the HAL as a separate layer. If your textbook has 4 layers, draw those and mention the HAL.

Topic 2: Android SDK (Software Development Kit)

Real-life example

To build furniture you need tools: saw, hammer, drill. The **Android SDK** is the **toolbox** for building Android apps: it contains the libraries, tools and a virtual phone to test on.

Definition

The **Android SDK** is a collection of libraries, tools, documentation and sample code that developers need to **build, test and debug** Android applications.

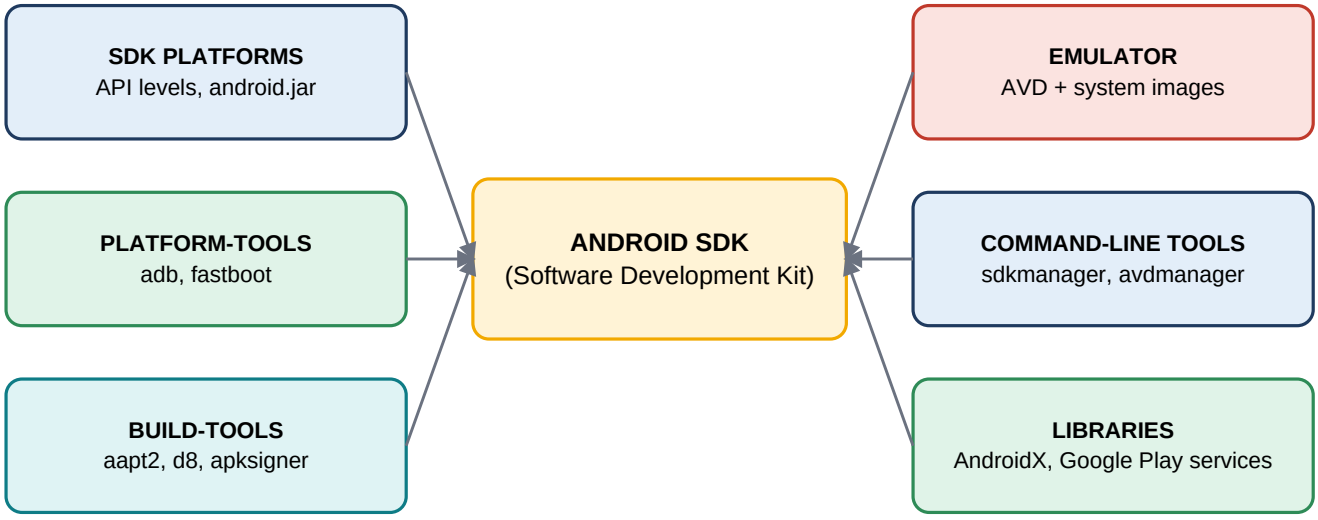


Figure 2: Main components of the Android SDK

Component	Purpose
SDK Platforms	The android.jar library for each API level; you compile your app against one platform
Platform-Tools	**adb** (Android Debug Bridge) to talk to a device, **fastboot** for flashing
Build-Tools	**aapt2** (resources), **d8** (DEX), **apksigner**, **zipalign**
Emulator and system images	Virtual phones (AVD) created from system images such as Android 14 x86_64
Command-line tools	sdkmanager (install packages), avdmanager (create AVDs)
SDK Manager	The window in the IDE to download platforms, tools and images
Libraries	AndroidX / Jetpack libraries, Google Play services

- **Also needed:** the **JDK** (Java Development Kit). Android Studio comes with its own JDK; for Eclipse you install the JDK yourself.
- **Logcat** shows system and app log messages for debugging. **DDMS** was the old debugging tool in Eclipse; its features are now part of Android Studio.
- **Key SDK settings: compileSdk** (API you compile against), **minSdk** (lowest Android your app runs on), **targetSdk** (version you tested for).

Topic 3: Installation of the Development Environment

3(a) Android Studio installation (current, official IDE)

> **Real-life example**
> Android Studio is like a fully equipped workshop in one box: editor, SDK, emulator and build tools come together.

- **Step 1:** Check the system requirements on **developer.android.com/studio** (a 64-bit OS, at least 8 GB RAM with 16 GB recommended, several GB of free disk space, hardware virtualization enabled for the emulator).
- **Step 2:** Download the Android Studio installer for your OS (Windows .exe, macOS .dmg, Linux .tar.gz).
- **Step 3:** Run the installer. Keep **Android Studio** and **Android Virtual Device** selected, choose the install folder and click Install.
- **Step 4:** On first launch the **Setup Wizard** opens. Choose the **Standard** setup, pick a theme and accept the licences.
- **Step 5:** The wizard downloads the **SDK, platform-tools and emulator**. Wait for it to finish and click Finish.
- **Step 6:** Open **Tools > SDK Manager** any time to install more platforms or tools.

3(b) Eclipse installation (as per syllabus; now discontinued)

> **Why 'discontinued'?**
> Google announced in 2015 that it would end Eclipse ADT support and formally ended it in 2016 after Android Studio 2.2. New SDK features do not work in Eclipse. Learn these steps for the exam, but use Android Studio for real work.

- **Step 1: Install the JDK** and set the **JAVA_HOME** environment variable.
- **Step 2: Download and unzip Eclipse** (Eclipse IDE for Java Developers). No installer is needed; run eclipse.exe.
- **Step 3: Download the Android SDK** (standalone SDK Tools) and open the **SDK Manager** to install the platform (for example Android 4.x), platform-tools and build-tools.
- **Step 4: Install the ADT plugin: Help > Install New Software > Add**, enter a name (ADT) and the ADT update-site URL, select **Developer Tools**, accept the licence and restart Eclipse.
- **Step 5: Point Eclipse to the SDK: Window > Preferences > Android**, set the **SDK Location**.
- **Step 6: Create an AVD: Window > AVD Manager > New**.
- **Step 7: Create a project: File > New > Android Application Project**.

Topic 4: Android Installation (running Android on an emulator or a phone)

Here 'Android installation' means getting an **Android device ready to run and test your app**: either a virtual device (emulator) or a real phone.

4(a) Create an AVD (Android Virtual Device) and use the emulator

> **Real-life example**
> An emulator is a **practice phone inside your computer**. You do not need to buy another phone to test the app.

- **1.** In Android Studio open **Tools > Device Manager** and click **Create Device**.
- **2.** Choose a **hardware profile** (for example Pixel) and click Next.
- **3.** Choose a **system image** (an Android version/API level; x86_64 images are faster on PCs). Download it if needed.
- **4.** Name the AVD, review the settings and click **Finish**.
- **5.** Press the **Play** button next to the AVD to start the emulator. Hardware acceleration (virtualization) must be enabled in the BIOS for good speed.

4(b) Run the app on a real Android phone

- **1. Enable Developer options:** Settings > About phone > tap **Build number** 7 times.
- **2. Turn on USB debugging:** Settings > Developer options > **USB debugging**.
- **3. Connect the phone** with a USB cable and tap **Allow** on the 'Allow USB debugging?' prompt.
- **4. Check the connection:** run `adb devices`; your device should be listed.
- **5. Run:** choose the phone in the device drop-down and press the green **Run** button.

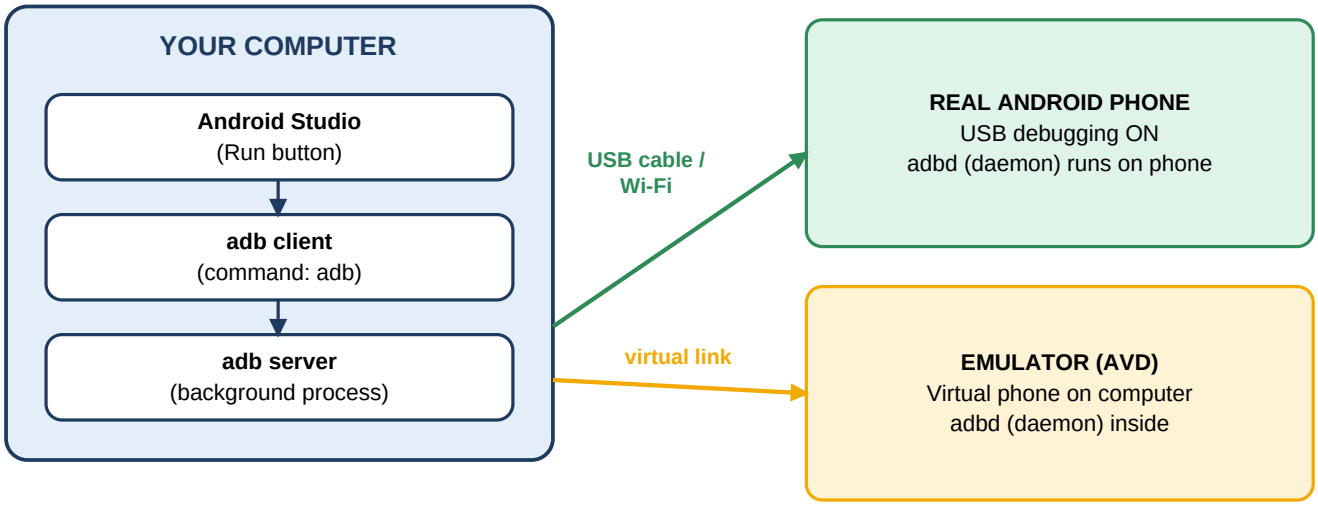


Figure 3: How the IDE talks to a device or emulator through adb

4(c) Installing an APK directly

- An **APK** (Android Package) is the installable app file. Install it with: `adb install app-debug.apk`.
- On a phone you can also copy the APK and open it, after allowing **Install unknown apps** for that file manager. Install apps only from trusted sources.

Topic 5: Building your First Android Application (Hello World)

Real-life example

The first program in every language prints 'Hello World'. It proves that your tools are installed correctly and shows the basic parts of an app.

Steps in Android Studio

- **1.** Open Android Studio and choose **New Project** (or File > New > New Project).
- **2.** Select the **Phone and Tablet** tab and the **Empty Views Activity** template (called 'Empty Activity' in older versions). Click Next.
- **3.** Enter the **Name** (HelloWorld), the **Package name** (com.example.helloworld), the save location, the **Language** (Java or Kotlin) and the **Minimum SDK**.
- **4.** Click **Finish**. Wait while **Gradle** builds the project (the first sync can take a few minutes).
- **5.** Open **res/layout/activity_main.xml** (the screen) and **MainActivity** (the code).
- **6.** Select a device (emulator or phone) and press the green **Run** button. 'Hello World!' appears on the screen.

MainActivity.java

```
package com.example.helloworld;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Understanding the code, line by line

Line	Meaning
`package com.example.helloworld;`	The package name: the unique address of your code.
`import ...AppCompatActivity;`	Brings in the base class for activities (AppCompatActivity gives modern look and backward compatibility).
`class MainActivity extends AppCompatActivity`	Our screen is an **Activity**. 'extends' means it inherits the features of AppCompatActivity.
`@Override`	We are replacing (overriding) a method of the parent class.
`onCreate(Bundle savedInstanceState)`	Called when the activity is first created. The Bundle holds saved state (for example after rotation).
`super.onCreate(savedInstanceState);`	Runs the parent class's setup code first. This line is required.
`setContentView(R.layout.activity_main);`	Shows the layout file activity_main.xml as the screen. R.layout.activity_main is the ID of that file.
`layout_width / layout_height`	**match_parent** = as big as the parent; **wrap_content** = only as big as the content.
`android:text="Hello World!"`	The text the TextView shows.

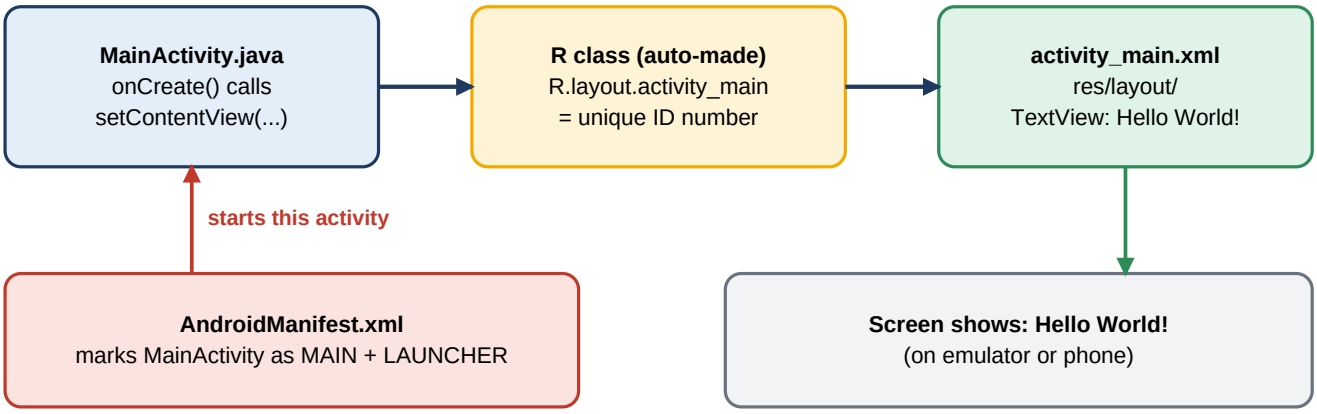


Figure 4: How the Manifest, MainActivity, R class and layout work together

Older (Eclipse) projects

In Eclipse projects the activity extended `Activity` (or ActionBarActivity) and the layout used `RelativeLayout`. The idea is exactly the same: `onCreate()` calls `setContentView(R.layout.activity_main)`.

Topic 6: Anatomy of an Android Application

> **Real-life example**
> The anatomy of a human body shows the parts (heart, brain, bones). The **anatomy of an Android app** shows the parts of a project: where the code, screens, images and settings live.

Android Studio project structure (Android view)

```
app                                  Main module of your app
  |-- manifests
      `-- AndroidManifest.xml        Declares components, permissions, app info
  |-- java
      `-- com.example.helloworld     Package name
          `-- MainActivity           Java / Kotlin source code (logic)
  `-- res
      |-- drawable                   Images and shapes
      |-- layout                     UI screens (activity_main.xml)
      |-- mipmap                     Launcher icons
      `-- values                     strings.xml, colors.xml, themes.xml
Gradle Scripts                       Build configuration
  |-- build.gradle (Project)         Settings for the whole project
  |-- build.gradle (Module: app)     App id, minSdk, targetSdk, dependencies
  |-- settings.gradle                Lists the modules
  |-- gradle.properties              Gradle options
  `-- local.properties               Local SDK path (not shared)
```

Figure 5: Project structure in Android Studio

Folder / file	What it contains
manifests / AndroidManifest.xml	The app's ID card: package, components, permissions, icon, theme
java (or kotlin)	Source code: activities, classes. Also test folders.
res/drawable	Images, icons and shape XML files
res/layout	XML files that design each screen
res/mipmap	Launcher icons in different sizes
res/values	strings.xml (text), colors.xml, themes.xml (older: styles.xml), dimens.xml
Gradle Scripts	Build files. In **build.gradle (Module: app)**: applicationId, minSdk, targetSdk, versionCode, versionName, dependencies

Classic Eclipse project structure (as in older textbooks)

Folder / file	Purpose
src	Java source files (MainActivity.java)
gen	Auto-generated files, mainly **R.java**. Never edit it.
Android x.x	The android.jar library of the chosen platform
assets	Raw files (fonts, HTML, text) kept as they are, accessed by file name
bin	Compiled output: classes.dex and the APK
libs	Extra third-party .jar libraries
res	Resources: drawable-hdpi/mdpi/xhdpi, layout, menu, values (strings.xml, styles.xml)
AndroidManifest.xml	Manifest file in the project root
project.properties / proguard	Target API setting / code shrinking and obfuscation rules

- **R.java / R class:** an auto-generated class that gives every resource a unique integer ID (for example `R.layout.activity_main`, `R.id.button1`, `R.string.app_name`). Your code uses it to find resources.
- **res vs assets:** files in **res** get an ID in R and are optimised by the system; files in **assets** are untouched and opened by name.
- **Why keep text in strings.xml?** It makes translation to other languages easy and avoids hard-coded text.

How an app is built: from source to APK

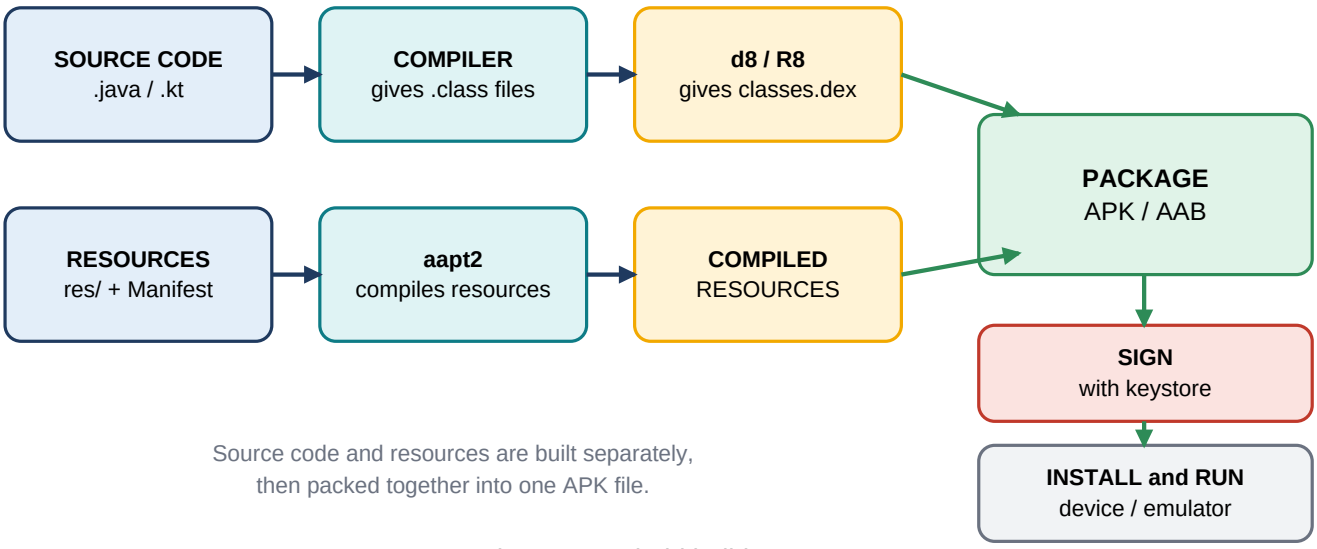


Figure 6: Android build process

- The compiler turns Java/Kotlin into **.class** files; **d8** converts them to **classes.dex** (Dalvik Executable) which ART can run.
- **aapt2** compiles resources and the manifest.
- Everything is packaged into an **APK** (or **AAB**, Android App Bundle) and **signed** with a key. Only a signed app can be installed. Google Play now requires AAB for new apps.

The four main components of an Android app

Component	Purpose	Example
Activity	One screen with a user interface	Login screen

Component	Purpose	Example
Service	Work in the background with no screen	Music playing
Broadcast Receiver	Responds to system or app messages	Battery low alert
Content Provider	Shares data between apps	Contacts data

These four components are declared in the manifest. They use **Intents** (messages) to start each other. They are covered in detail in Unit II.

Topic 7: AndroidManifest.xml (Android Manifest file)

Real-life example

Every person has an **ID card**: name, address, what they are allowed to do. The **manifest** is the ID card of an app. Android reads it **before running any code** to learn what the app is and what it needs.

Definition

The **AndroidManifest.xml** is a mandatory XML file in the root of every Android app that **describes the app to the Android system**: its package name, components, permissions, hardware needs and more.

What the manifest does

- Names the app's **package** (unique identity).
- **Declares every component** (activities, services, receivers, providers). A component not declared here cannot run.
- Lists the **permissions** the app needs (INTERNET, CAMERA, READ_CONTACTS) and permissions others need to use it.
- Says which activity is the **launcher** (entry point) using an **intent filter**.
- Declares needed **hardware/software features** and the app's icon, label and theme.

AndroidManifest.xml (example)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.helloworld">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.HelloWorld">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>
</manifest>
```

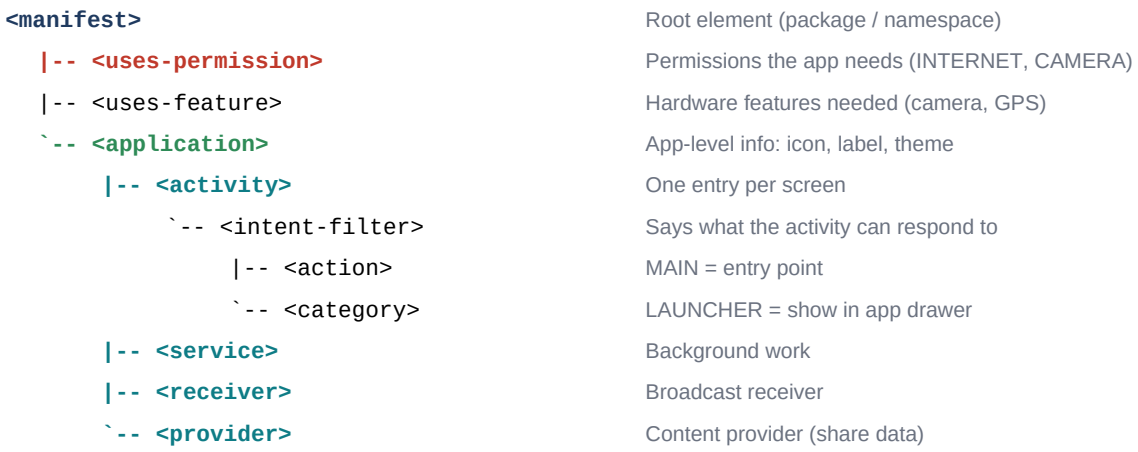


Figure 7: Structure of AndroidManifest.xml

Element / attribute	Meaning
<manifest>	Root element. The **package** attribute is the app's unique name.
<uses-permission>	Asks permission, e.g. android.permission.INTERNET
<uses-feature>	Hardware needed (camera, GPS); Play Store hides the app from phones without it
<application>	Container for components; attributes icon, label, theme, allowBackup
<activity>	Declares one screen. **android:name** is its class (.MainActivity).
<service>, <receiver>, <provider>	Declare a service, broadcast receiver and content provider
<intent-filter>	Tells what intents a component can handle (action, category, data)
MAIN + LAUNCHER	MAIN = entry point of the app; LAUNCHER = show the icon in the app drawer
android:exported	Whether other apps can start the component. Required when a component has an intent filter and the app targets Android 12 (API 31) or higher.

Good to know

In older projects and most textbooks, the **package** and **<uses-sdk>** (minSdkVersion, targetSdkVersion) are written in the manifest. In new Android Studio projects, the package is set as **namespace** and the SDK versions are set in **build.gradle**, so they are not in the manifest. For the exam, write them as the textbook shows.

PART B: Important Questions with Model Answers (English)

Priority: **Very Likely** = appears most often in BTEUP. **Likely** = good chance. **Possible** = can appear. Each answer follows the exam format: **Definition, Points, Diagram/Table, Conclusion**.

Q1. What is Android? Explain its features and versions. VERY LIKELY

Definition / Introduction: Android is an open-source, Linux-based mobile operating system developed by Google and the Open Handset Alliance. It is used on smartphones, tablets, smart TVs, watches and cars.

Points

- **Open source and free:** manufacturers can customise it.
- **Rich user interface:** widgets, notifications, multitasking.
- **Connectivity:** Wi-Fi, Bluetooth, NFC, GPS, mobile data.
- **Storage:** SQLite, files, shared preferences. **Hardware access:** camera, sensors.
- **Development:** apps are written in Java or Kotlin and published on Google Play.
- **Security:** each app runs in its own sandbox and needs permissions.
- **Versions:** named after desserts until Android 9 (Cupcake, Donut, Eclair, Froyo, Gingerbread, Honeycomb, Ice Cream Sandwich, Jelly Bean, KitKat, Lollipop, Marshmallow, Nougat, Oreo, Pie); numbered from Android 10.

Version	Name	API level
1.5	Cupcake	3
4.4	KitKat	19
5.0	Lollipop	21
6.0	Marshmallow	23
8.0	Oreo	26
9	Pie	28
10	(no name)	29

Conclusion: Android is the most widely used mobile platform because it is open, flexible and supported by many manufacturers.

Q2. Explain the architecture of Android with a diagram. VERY LIKELY

Definition / Introduction: The Android platform is a software stack arranged in layers. Each layer provides services to the layer above it.

Points

- **Linux kernel:** the foundation. Provides drivers, memory, process and power management, and security.
- **Hardware Abstraction Layer (HAL):** standard interfaces to hardware such as camera, audio, Bluetooth and sensors.
- **Android Runtime (ART):** runs the app's DEX bytecode; each app has its own process and ART instance; includes core Java libraries (Dalvik before Android 5.0).
- **Native C/C++ libraries:** SQLite, OpenGL ES, SSL, libc and media libraries.
- **Java API framework:** Activity Manager, View System, Content Providers, Resource Manager and Notification Manager.
- **System apps:** built-in apps such as phone, contacts, browser and email; user apps run at this level.

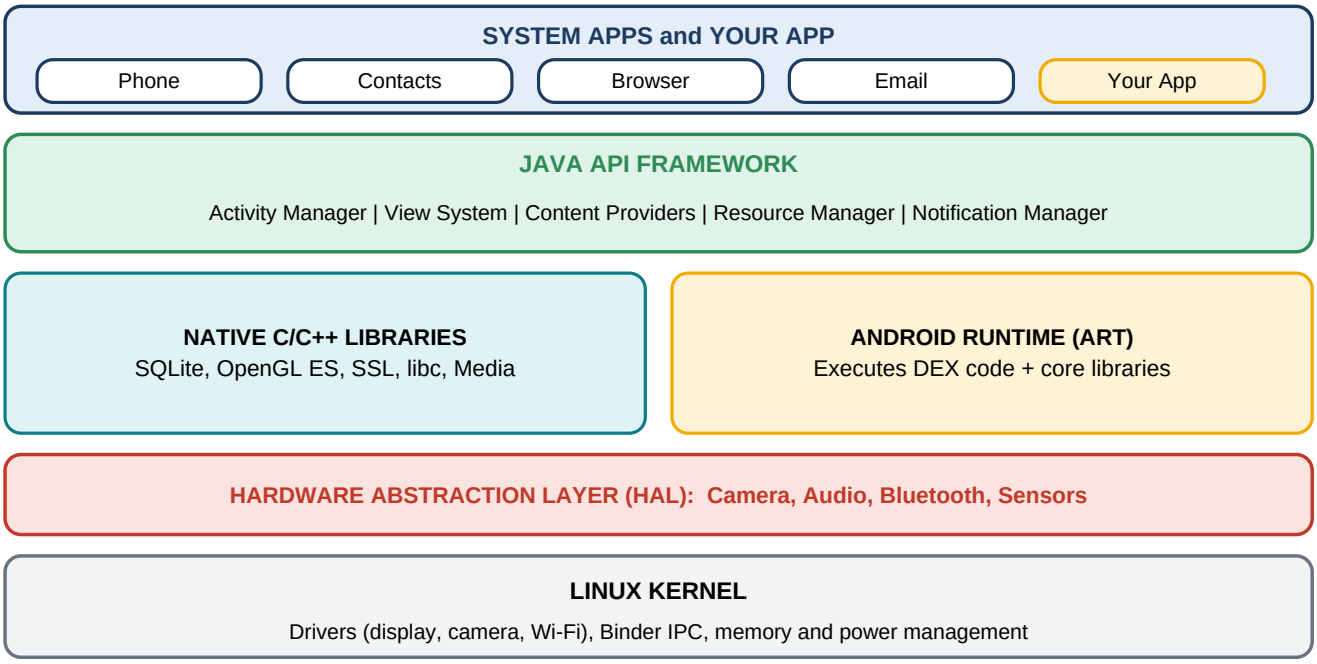


Figure: Android software stack

Conclusion: The layered design makes Android portable across hardware and lets developers use high-level APIs without handling hardware details.

Q3. What is the Android SDK? Explain its components. VERY LIKELY

Definition / Introduction: The Android SDK (Software Development Kit) is a set of libraries, tools, documentation and sample code needed to build, test and debug Android applications.

Points

- **SDK Platforms:** android.jar for each API level.
- **Platform-Tools:** adb (Android Debug Bridge) and fastboot.
- **Build-Tools:** aapt2, d8, apksigner, zipalign to build and sign the APK.
- **Emulator and system images:** to create Android Virtual Devices (AVD).
- **SDK Manager and command-line tools:** download and manage packages (sdkmanager, avdmanager).
- **Libraries and tools:** AndroidX, Google Play services, Logcat for debugging.

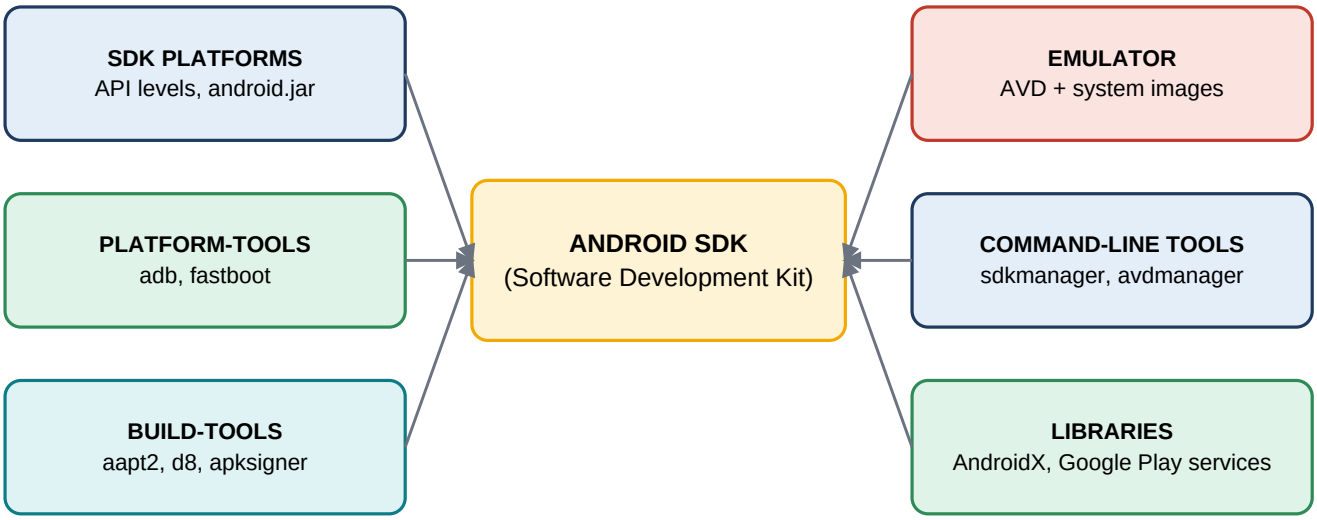


Figure: Android SDK components

Conclusion: The SDK, together with the JDK and an IDE, is everything required to develop Android apps.

Q4. Explain the steps to install Android Studio. LIKELY

Definition / Introduction: Android Studio is the official Integrated Development Environment (IDE) for Android. It includes the code editor, SDK, emulator and build tools.

Points

- Check the system requirements (64-bit OS, at least 8 GB RAM with 16 GB recommended, enough disk space, virtualization enabled).
- Download the installer from developer.android.com/studio for Windows, macOS or Linux.
- Run the installer, keep Android Studio and Android Virtual Device selected, choose the folder and install.
- Start Android Studio. In the Setup Wizard choose the Standard setup, a theme, and accept the licences.
- The wizard downloads the SDK, platform-tools and emulator. Click Finish.
- Use Tools > SDK Manager to add more platforms or tools later.

Conclusion: After installation, a new project can be created and run on an emulator or a real phone.

Q5. Explain the installation of Eclipse and the ADT plugin for Android development. LIKELY

Definition / Introduction: Before Android Studio, Android apps were developed in the Eclipse IDE using the Android Developer Tools (ADT) plugin. Google ended ADT support in 2016, but the procedure is part of the syllabus.

Points

- **Install the JDK** and set JAVA_HOME.
- **Download and unzip Eclipse** (Eclipse IDE for Java Developers) and run it.
- **Download the Android SDK** and use the SDK Manager to install a platform, platform-tools and build-tools.
- **Install the ADT plugin:** Help > Install New Software > Add, give the ADT update-site URL, select Developer Tools, accept the licence and restart.
- **Configure:** Window > Preferences > Android > set the SDK Location.
- **Create an AVD** from Window > AVD Manager, then create a project with File > New > Android Application Project.

Conclusion: Eclipse with ADT is now obsolete; Android Studio is the official and recommended IDE.

Q6. How do you create an AVD and run an app on an emulator and on a real device?

LIKELY

Definition / Introduction: An AVD (Android Virtual Device) is a virtual phone running on the computer, used to test apps without a real device.

Points

- **Create AVD:** Tools > Device Manager > Create Device > choose a hardware profile > choose a system image (API level) > name it > Finish.
- **Run on emulator:** start the AVD with the Play button, select it in the device list and press Run.
- **Real device:** enable Developer options (tap Build number 7 times), turn on USB debugging, connect with USB and allow the prompt.
- Check with `adb devices`, select the phone and press Run.
- An APK can also be installed directly with `adb install app.apk`.

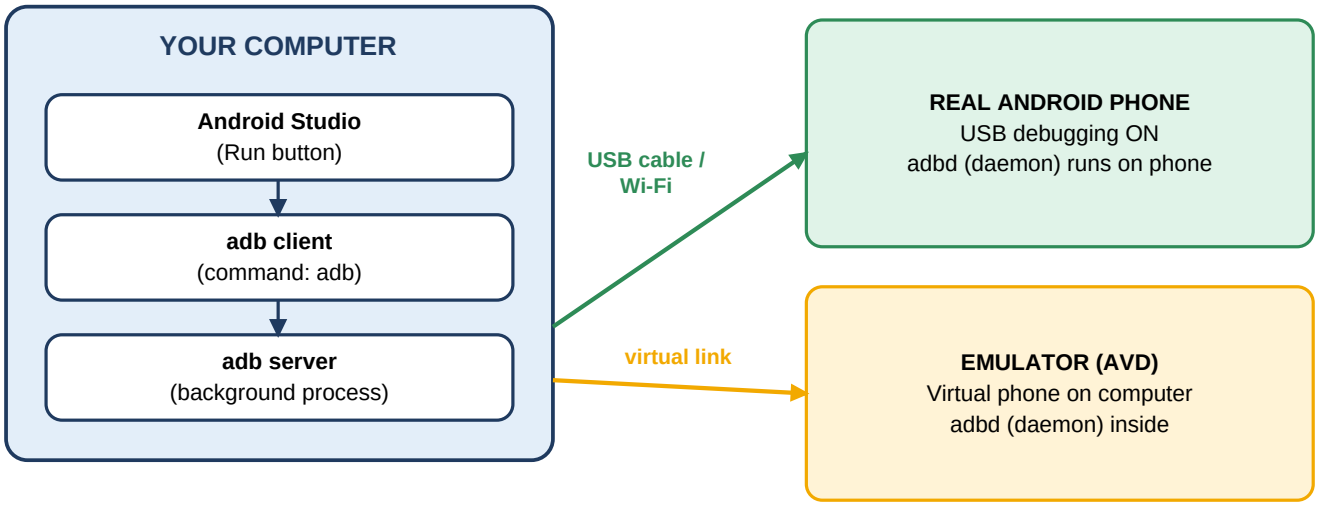


Figure: IDE, adb and device

Conclusion: The emulator is convenient for testing different devices, while a real phone gives the true speed and behaviour.

Q7. Write the steps to build your first Android application (Hello World). VERY LIKELY

Definition / Introduction: The Hello World application is the simplest Android app. It displays the text 'Hello World!' on the screen and verifies that the development environment works.

Points

- Open Android Studio > **New Project** > choose **Empty Views Activity** > Next.
- Enter the app name (HelloWorld), package name (com.example.helloworld), save location, language (Java/Kotlin) and minimum SDK > **Finish**.
- Wait for the **Gradle** build to finish.
- **MainActivity** contains the code; `onCreate()` calls `setContentView(R.layout.activity_main)` to show the layout.
- **activity_main.xml** holds a **TextView** with `android:text="Hello World!"`.
- Select an emulator or a phone and press **Run**.

MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

activity_main.xml (TextView)

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />
```

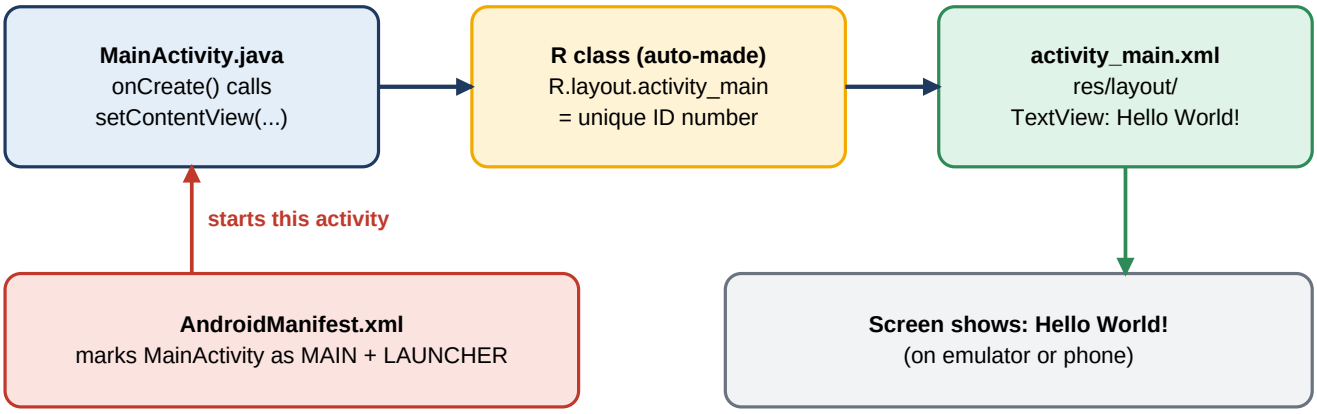


Figure: How the parts of the app are connected

Conclusion: The Hello World app shows the basic parts of every Android app: an activity, a layout and the manifest.

Q8. Explain the anatomy of an Android application (project structure). VERY LIKELY

Definition / Introduction: The anatomy of an Android application describes the folders and files of an Android project and the purpose of each.

Points

- **manifests:** AndroidManifest.xml describes the app to the system.
- **java (or kotlin):** the source code, such as MainActivity.
- **res:** resources: drawable (images), layout (screens), mipmap (launcher icons), values (strings.xml, colors.xml, themes.xml).
- **Gradle Scripts:** build.gradle files that set applicationId, minSdk, targetSdk, versions and dependencies.
- **R class:** auto-generated class giving each resource a unique ID.
- **In Eclipse:** src, gen (R.java), Android library, assets, bin, libs, res and AndroidManifest.xml.

```
app                                  Main module of your app
  |-- manifests
       `-- AndroidManifest.xml       Declares components, permissions, app info
  |-- java
       `-- com.example.helloworld    Package name
            `-- MainActivity         Java / Kotlin source code (logic)
  `-- res
       |-- drawable                  Images and shapes
       |-- layout                    UI screens (activity_main.xml)
       |-- mipmap                    Launcher icons
       `-- values                    strings.xml, colors.xml, themes.xml
Gradle Scripts                       Build configuration
  |-- build.gradle (Project)         Settings for the whole project
  |-- build.gradle (Module: app)     App id, minSdk, targetSdk, dependencies
  |-- settings.gradle                Lists the modules
  |-- gradle.properties              Gradle options
  `-- local.properties               Local SDK path (not shared)
```

Figure: Android Studio project structure

Conclusion: Knowing where each file belongs makes it easy to build and maintain an Android app.

Q9. Explain the Android build process. What are R.java and the resources folder?

LIKELY

Definition / Introduction: The build process converts source code and resources into an installable, signed APK or AAB file.

Points

- Java/Kotlin code is compiled into **.class** files and converted by **d8** into **classes.dex**.
- **aapt2** compiles the resources and manifest.
- All parts are packaged into an **APK** (or **AAB**) and signed with a keystore.
- **R.java (R class):** auto-generated; maps each resource to a unique integer ID, e.g. R.layout.activity_main, R.id.button1. It must not be edited.
- **res folder:** holds resources (drawable, layout, values, mipmap, menu). Keeping text in strings.xml supports translation.
- **assets folder:** raw files accessed by name, with no ID in R.

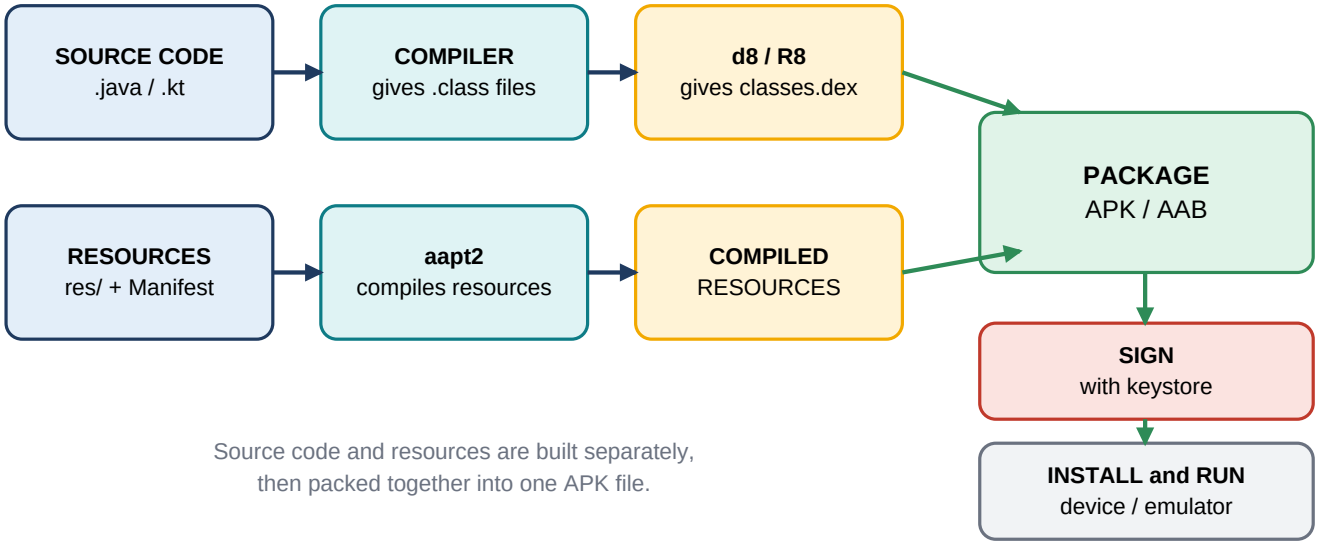


Figure: Android build process

Conclusion: The build system automates compiling, packaging and signing so the developer can focus on the app.

Q10. What is AndroidManifest.xml? Explain its structure and important elements.

VERY LIKELY

Definition / Introduction: AndroidManifest.xml is a mandatory XML file in the root of every Android app. It describes the app to the Android system, which reads it before running any code.

Points

- **<manifest>:** root element; the package attribute gives the unique app name.
- **<uses-permission>:** permissions the app needs, e.g. INTERNET, CAMERA.
- **<uses-feature>:** hardware features the app needs.
- **<application>:** app-level settings (icon, label, theme) and it contains the components.
- **<activity>, <service>, <receiver>, <provider>:** declare the four components.
- **<intent-filter>** with action MAIN and category LAUNCHER marks the launcher activity.

```
<manifest>                          Root element (package / namespace)
  |-- <uses-permission>             Permissions the app needs (INTERNET, CAMERA)
  |-- <uses-feature>                Hardware features needed (camera, GPS)
  `-- <application>                 App-level info: icon, label, theme
      |-- <activity>                One entry per screen
          `-- <intent-filter>       Says what the activity can respond to
              |-- <action>          MAIN = entry point
              `-- <category>        LAUNCHER = show in app drawer
      |-- <service>                 Background work
      |-- <receiver>                Broadcast receiver
      `-- <provider>                Content provider (share data)
```

Figure: Structure of AndroidManifest.xml

Example manifest

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.helloworld">
    <uses-permission android:name="android.permission.INTERNET" />
    <application android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name">
        <activity android:name=".MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Conclusion: Without a correct manifest the app cannot be installed or run, because the system would not know its components or permissions.

Q11. Explain the components of an Android application. LIKELY

Definition / Introduction: An Android application is built from four types of components. Each is declared in the manifest and can be started by the system.

Points

- **Activity:** a single screen with a user interface, e.g. a login screen.
- **Service:** runs in the background without a UI, e.g. playing music.
- **Broadcast Receiver:** responds to system-wide messages, e.g. battery low.
- **Content Provider:** shares data between apps, e.g. contacts.
- **Intents** are messages used to start components.

Component	Purpose	Example
Activity	One screen with UI	Login screen
Service	Background work	Music playing
Broadcast Receiver	React to messages	Battery low
Content Provider	Share data	Contacts

Conclusion: These components make an app modular, and each can be reused by other apps through intents.

Q12. Differentiate between Dalvik and ART. What is the difference between APK and AAB? POSSIBLE

Definition / Introduction: Dalvik and ART are the runtimes that execute Android apps. ART replaced Dalvik as the default runtime in Android 5.0 (API level 21).

Points

- Both run apps in **DEX** (Dalvik Executable) format.
- **Dalvik** compiled code at run time (just-in-time), which used more CPU and battery. **ART** compiles code ahead of time at install, so apps start faster; newer versions combine AOT, JIT and profile-guided compilation.
- **APK** (Android Package) is the installable file that contains everything for all devices. **AAB** (Android App Bundle) is a publishing format uploaded to Google Play, which then builds optimised APKs for each device; Google Play requires AAB for new apps.

Point	Dalvik	ART
Compilation	Just-in-time (at run time)	Ahead-of-time at install (plus JIT, profile-guided in newer versions)
App start	Slower	Faster

Point	APK	AAB
Purpose	Installable file	Publishing format for Google Play
Installable directly	Yes	No (Google Play builds APKs from it)
Size on device	Larger (all resources)	Smaller (only what the device needs)

Conclusion: ART gives better performance than Dalvik, and AAB gives smaller downloads than a single universal APK.

Quick Revision Sheet (read the night before)

Term	Remember this
Android	Open-source, Linux-based OS by Google + OHA. First phone HTC Dream (2008)
Architecture	Linux kernel > HAL > ART + C/C++ libraries > Java API framework > System apps
ART / Dalvik	ART default from Android 5.0 (API 21); runs DEX
Android SDK	Platforms, Platform-Tools (adb), Build-Tools, Emulator, Command-line tools
API level	Number for each Android version. minSdk, targetSdk, compileSdk
Android Studio	Official IDE. Eclipse ADT support ended 2016
Eclipse setup	JDK > Eclipse > SDK > ADT plugin > SDK path > AVD
AVD	Virtual phone. Tools > Device Manager > Create Device
Real phone	Tap Build number 7 times > USB debugging ON > adb devices
adb	Android Debug Bridge. adb install app.apk
First app	Empty Views Activity > Name, Package, Language, Min SDK > Finish > Run
onCreate()	Called when the activity is created; calls setContentView()
Project folders	manifests, java, res (drawable, layout, mipmap, values), Gradle Scripts
R class	Auto-generated IDs for resources. Never edit
res vs assets	res gets IDs; assets are raw files opened by name
Build	Source > .class > d8 > classes.dex; resources > aapt2; package APK/AAB; sign
4 components	Activity, Service, Broadcast Receiver, Content Provider
Manifest	Root XML. package, uses-permission, application, activity, intent-filter (MAIN, LAUNCHER)
android:exported	Needed with intent-filter when targeting Android 12+ (API 31)

Memory tricks

Architecture (bottom to top): L-H-R-J-S = Linux, HAL, Runtime (and libraries), Java framework, System apps.
4 components: A-S-B-C = Activity, Service, Broadcast receiver, Content provider.
res folders: D-L-M-V = drawable, layout, mipmap, values.
Manifest launcher: MAIN = entry point, **LAUNCHER** = icon on the home screen.

Common Mistakes Students Make

- Saying Eclipse is the official IDE. **Android Studio** is official now; Eclipse ADT support ended in 2016.
- Forgetting that the manifest **declares all components**. A component missing from the manifest will not run.
- Editing **R.java** by hand. It is auto-generated.
- Mixing up **res** (resources with IDs) and **assets** (raw files).
- Not drawing the architecture diagram and project tree. They carry marks.

Self-Test (answer without looking, 5 marks each)

- Draw and explain the architecture of Android.
- What is the Android SDK? Write any four components.
- Write the steps to create and run a Hello World application.
- What is AndroidManifest.xml? Write any four elements.
- Name the four components of an Android application with one example each.

Self-Test Answers (check after writing)

1. Layers from bottom: Linux kernel, HAL, Android Runtime (ART) and native C/C++ libraries, Java API framework, System apps. Explain each in one line. Draw Figure 1.

2. The SDK is a set of libraries, tools, documentation and samples to build, test and debug apps. Components: SDK Platforms, Platform-Tools (adb), Build-Tools, Emulator and system images, command-line tools, libraries.

3. New Project > Empty Views Activity > enter name, package, language, minimum SDK > Finish > wait for Gradle > edit MainActivity and activity_main.xml > choose emulator or phone > Run.

4. The manifest is the XML file that describes the app to the Android system. Elements: manifest, uses-permission, uses-feature, application, activity, service, receiver, provider, intent-filter.

5. Activity (login screen), Service (music playing), Broadcast Receiver (battery low alert), Content Provider (contacts data).

> **Done with Unit I!**
> If you scored 4 out of 5 or more, move to **Unit II**. If not, re-read the topics you missed in Part A.